Porte de poulailler automatique

Rétrofit Omlet Autodoor — Contrôleur ESP32

Documentation technique v2 — Mars 2026

Projet GitHub : https://github.com/bonnoa/retrofit_omlet_automatic_door

1. Présentation du projet

Ce projet consiste à remplacer l'électronique d'origine d'une porte automatique Omlet Autodoor par un contrôleur basé sur un ESP32, tout en conservant la mécanique existante (moteur, rail, porte coulissante).

La porte coulisse horizontalement : de droite vers gauche pour s'ouvrir, et de gauche vers droite pour se fermer. Le moteur d'origine Omlet (014.0077) fonctionne à 9V optimal / 12V maximum.

1.1 Objectifs

- 4 modes de gestion : soleil (lever/coucher), luminosité (seuils lux), heure fixe, manuel
- Intégration Home Assistant via MQTT
- Interface web mobile-first embarquée sur l'ESP32 (WebSocket)
- Détection de blocage par surcourant (ACS712) et timeout
- Arrêt précis par fins de course magnétiques (MC-38)
- Persistance des paramètres entre redémarrages (NVS Flash)

2. Liste des composants

Composant	Référence / Description	Rôle
Microcontrôleur	ESP32-WROOM-32U DevKitC	Cerveau + WiFi + serveur web
Contrôleur moteur	BTS7960 / IBT-2	Pilotage moteur bidirectionnel PWM
Fins de course	MC-38 NF × 2	Détection position ouverte/fermée
Capteur courant	ACS712 5A	Détection surcourant / blocage
Capteur luminosité	BH1750 (GY-30)	Mode luminosité + log
Convertisseur tension	LM2596S (ajustable)	12V → 5V alimentation ESP32
Alimentation	12V 2A (KDT-1220 ou équiv.)	Alimentation principale
Boîtier	190×150×75 mm IP67 ABS	Protection étanche extérieure
Câble signal	Câble alarme 4×0.22mm²	FDC et BH1750 vers boîtier
Presse-étoupes	M16 ou M20	Entrées câbles sur boîtier
Entretoises	M3 nylon 10mm	Fixation modules sur plaque

Note : le LM2596S doit être calibré à 5V avant branchement. Alimenter l'ESP32 via USB indépendant si chutes de tension au démarrage moteur.

3. Schéma de câblage électronique

Le schéma complet est disponible dans le fichier schema_porte_poulailler.svg joint au projet.

3.1 Affectation des pins ESP32

GPIO	Composant	Signal	Description
GPIO 25	BTS7960	RPWM	PWM ouverture (droite → gauche)
GPIO 26	BTS7960	LPWM	PWM fermeture (gauche → droite)
GPIO 27	BTS7960	EN	Enable R_EN + L_EN (reliés ensemble)
GPIO 32	MC-38 gauche	FDC_OUV	Fin de course position ouverte
GPIO 33	MC-38 droit	FDC_FER	Fin de course position fermée
GPIO 34	ACS712	OUT	Mesure courant (entrée analogique)
GPIO 21	BH1750	SDA	I2C données
GPIO 22	BH1750	SCL	I2C horloge
VIN / 5V	LM2596S	5V	Alimentation ESP32
GND	Tous	GND	Masse commune obligatoire

3.2 BTS7960

Broche BTS7960	Connecter à
B+ (12V)	12V alimentation
B− (GND)	GND commun
VCC	3.3V ESP32
R_EN + L_EN	GPIO27 (reliés ensemble sur même fil)
RPWM	GPIO25
LPWM	GPIO26
OUT+ (M+)	ACS712 IP+
OUT− (M−)	Moteur fil noir (−)

3.3 ACS712

Broche ACS712	Connecter à
VCC	5V (LM2596S sortie)
GND	GND commun
OUT	GPIO34 (ESP32)
IP+	BTS7960 OUT+
IP−	Moteur fil rouge (+)

3.4 Fins de course MC-38

Les MC-38 sont des interrupteurs reed en mode NO (Normalement Ouvert) : aimant présent = contact fermé = pin LOW.

Capteur	Fil 1	Fil 2	Position
MC-38 gauche	GPIO32	GND	Porte ouverte
MC-38 droit	GPIO33	GND	Porte fermée

Les deux fils de chaque capteur sont interchangeables (pas de polarité). Utiliser un câble alarme 4 fils pour passer les deux capteurs dans le même presse-étoupe.

3.5 BH1750

Broche BH1750	Connecter à
VCC	3.3V (ESP32)
GND	GND commun
SCL	GPIO22
SDA / DAT	GPIO21
ADDR	Non connecté (adresse 0x23)

Le BH1750 est monté à l'extérieur du boîtier, orienté vers le ciel.

3.6 LM2596S — Calibration

Avant tout branchement, régler la tension de sortie à 5,0V :

- Alimenter le module en 12V
- Mesurer la tension de sortie avec un multimètre
- Tourner le potentiomètre jusqu'à obtenir exactement 5,0V
- Vérifier la stabilité sous charge (résistance 100Ω en test)

4. Logiciel

4.1 Structure des fichiers

Fichier	Rôle
porte_poulailler.ino	Fichier principal : logique moteur, modes, MQTT, NVS
web_server_v2.ino	Serveur web AsyncWebServer + WebSocket + HTML embarqué

Les deux fichiers doivent être dans le même dossier nommé porte_poulailler. L'IDE Arduino les compile automatiquement ensemble.

4.2 Dépendances Arduino

Bibliothèque	Auteur	Installation
PubSubClient	Nick O'Leary	Gestionnaire de bibliothèques Arduino
ArduinoJson	Benoit Blanchon	Gestionnaire de bibliothèques Arduino
BH1750	Christopher Laws	Gestionnaire de bibliothèques Arduino
ESP Async WebServer	ESP32Async	github.com/ESP32Async/ESPAsyncWebServer
AsyncTCP	ESP32Async	github.com/ESP32Async/AsyncTCP
Preferences	(intégrée ESP32)	Incluse dans le package ESP32

4.3 Configuration Arduino IDE

Package ESP32 : ajouter l'URL suivante dans Fichier → Préférences → URLs de gestionnaire de cartes supplémentaires :

`https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`

Paramètre	Valeur
Board	ESP32 Dev Module
Upload Speed	921600
CPU Frequency	240MHz (WiFi/BT)
Flash Frequency	80MHz
Flash Mode	DIO
Flash Size	4MB (32Mb)
Partition Scheme	Default 4MB with spiffs

Paramètre	Valeur
PSRAM	Disabled

4.4 Paramètres à configurer dans le code

Constante	Valeur par défaut	Description
WIFI_SSID	"TON_SSID"	Nom du réseau WiFi
WIFI_PASSWORD	"TON_MOT_DE_PASSE"	Mot de passe WiFi
MQTT_SERVER	"192.168.1.XX"	Adresse IP du broker MQTT (Home Assistant)
MQTT_PORT	1883	Port MQTT
MQTT_USER / MQTT_PASS	—	Identifiants MQTT
LATITUDE / LONGITUDE	48.8566 / 2.3522	Coordonnées GPS pour calcul soleil
ACS712_ZERO_ADC	3050	Valeur ADC à 0A — à calibrer
ACS712_SEUIL_ADC	310	Delta ADC déclenchant l'arrêt (~2.7A)
PWM_DUTY_9V	191	Duty cycle ouverture (~9V sur 12V)
PWM_DUTY_FER	140	Duty cycle fermeture (~7V, plus doux)
TIMEOUT_OUV_MS	15000	Timeout ouverture en ms
TIMEOUT_FER_MS	20000	Timeout fermeture en ms
RAMPE_MS	300	Durée de la rampe PWM au démarrage

5. Modes de gestion

Mode	Déclenchement	Paramètres configurables
Soleil	Lever/coucher calculés (NOAA) + offsets	offsetOuv, offsetFer (−60 à +120 min)
Luminosité	Seuils lux + durée de confirmation	luxSeuilOuverture, luxSeuilFermeture, luxConfirmationSec
Heure fixe	Heures définies + offsets optionnels	heureFixeOuv/Fer H:M + offsetFixeOuv/Fer
Manuel	Commandes manuelles uniquement	—

Le mode actuel et tous les paramètres sont persistés en mémoire flash NVS — ils survivent aux redémarrages de l'ESP32.

Un override manuel est possible en mode automatique : une commande manuelle suspend temporairement l'automatisme jusqu'à la prochaine échéance (prochain lever/coucher ou 30 minutes en mode luminosité).

6. Topics MQTT

Topic	Direction	Contenu / Valeurs
poulailler/porte/etat	Publish	open / closed / opening / closing / error
poulailler/porte/commande	Subscribe	OPEN / CLOSE / STOP / RESET
poulailler/capteurs/lux	Publish	Valeur float en lux
poulailler/capteurs/courant	Publish	Valeur float en ampères
poulailler/soleil/heures	Publish	JSON : lever, coucher, ouverture, fermeture
poulailler/mode	Pub/Sub	soleil / luminosite / heure_fixe / manuel
poulailler/config	Subscribe	JSON complet de configuration

7. Logique de sécurité

Protection	Condition	Action
Fins de course	FDC atteint pendant mouvement	Arrêt moteur immédiat, état mis à jour
Timeout ouverture	FDC non atteint après 15s	Arrêt moteur + état ERREUR
Timeout fermeture	FDC non atteint après 20s	Arrêt moteur + état ERREUR
Surcourant ACS712	Delta ADC > 310 (~2.7A)	Arrêt moteur + état ERREUR
Garde universelle	Moteur actif hors mouvement	PWM forcé à 0 à chaque cycle loop()
Reset erreur	Bouton web ou commande RESET	Relit les FDC, remet l'état à jour

8. Interface web embarquée

L'interface web est accessible depuis le navigateur du téléphone à l'adresse http://[IP_ESP32]. L'IP s'affiche dans le moniteur série au démarrage.

- État de la porte en temps réel (WebSocket, mise à jour toutes les 2s)
- Boutons Ouvrir / Fermer / Stop d'urgence (double confirmation)
- 4 onglets de configuration de mode avec panneaux dédiés
- Graphique courant moteur, journal des événements
- Bouton Réinitialiser l'erreur (visible uniquement en état ERREUR)
- Heure ESP32 affichée dans le header (contrôle NTP/fuseau horaire)

L'heure été/hiver est gérée automatiquement via la chaîne POSIX : CET-1CEST,M3.5.0,M10.5.0/3 (France). Aucune modification manuelle n'est nécessaire au changement d'heure.

9. Installation et mise en service

9.1 Première mise en service

1. Calibrer le LM2596S à 5V avant tout branchement
2. Modifier les paramètres WiFi, MQTT et GPS dans porte_poulailler.ino
3. Installer les bibliothèques Arduino requises
4. Compiler et téléverser les deux fichiers .ino dans le même dossier porte_poulailler
5. Ouvrir le moniteur série (115200 bauds) pour vérifier la connexion WiFi et NTP
6. Accéder à l'interface web à http://[IP_ESP32]

9.2 Calibration ACS712

- Téléverser le sketch de test fourni dans le guide (lecture brute GPIO34)
- Moteur débranché : noter la valeur RAW moyenne → c'est ACS712_ZERO_ADC
- Moteur en marche normale : noter le delta max → seuil = delta max × 1.5
- Ajuster ACS712_ZERO_ADC et ACS712_SEUIL_ADC dans le code

9.3 Positionnement des fins de course

- MC-38 gauche : fixé sur le cadre, aimant sur la porte côté position ouverte (extrémité gauche)
- MC-38 droit : fixé sur le cadre, aimant sur la porte côté position fermée (extrémité droite)
- Vérifier : aimant face au capteur = pin LOW (contact fermé)
- Vérifier via moniteur série : [INIT] Porte OUVERTE ou [INIT] Porte FERMÉE au démarrage

10. Dépannage rapide

Symptôme	Cause probable	Solution
[BH1750] Device is not configured!	Mauvais câblage I2C ou alimentation	Vérifier SDA→21, SCL→22, VCC→3.3V, GND
ESP32 reboot au démarrage moteur	Chute tension 5V LM2596S	Condensateur 470µF sur sortie 5V ou alim USB séparée
Porte ne s'arrête pas sur FDC	Logique HIGH/LOW inversée	Vérifier type MC-38 (NO/NF) et câblage
Surcourant intempestif	ACS712_ZERO_ADC mal calibré	Recalibrer avec sketch de test à moteur arrêté
Boucle MQTT mode	Message retain en conflit	Purger le retain : publier payload vide sur poulailler/mode
[INIT] Position inconnue	Aucun FDC actif au	Déplacer la porte manuellement sur un

Symptôme	Cause probable	Solution
	démarrage	FDC puis redémarrer
Interface web inaccessible	IP changée	Vérifier l'IP dans le moniteur série ou fixer en DHCP statique